```
import os

# Must be set before pygame/config import.
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")
os.environ.setdefault("SDL_AUDIODRIVER", "dummy")
os.environ.setdefault("PYGAME_HIDE_SUPPORT_PROMPT", "1")

import importlib
import importlib.util
import traceback

import pygame

import config
import game_core as gc
import menu

# Set this to True after you finish splitting game_core.py into modules.
# If False, missing split modules are reported as SKIP instead of FAIL.
REQUIRE_SPLIT_MODULES = True

VERBOSE_TRACEBACKS = True

SPLIT_MODULES = [
    "state",
    "player",
    "weapons",
    "enemies",
    "cutscenes",
    "watching_ads",
    "effects",
    "ui_hud",
    "interactions",
]

class EmptyKeys:
    """Fake pygame.key.get_pressed() object.

    Any key lookup returns False.
    """
    def __getitem__(self, key):
        return False

def run_check(name, fn, required=True):
    try:
        fn()
        pygame.event.pump()

        if pygame.display.get_surface() is not None:
            pygame.display.flip()

    except Exception as e:
        status = "FAIL" if required else "WARN"
        print(f"[{name}] {status}: {e!r}")

        if required and VERBOSE_TRACEBACKS:
            traceback.print_exc()

        return not required

    print(f"[{name}] OK")
    return True

def run_import_check(module_name):
    spec = importlib.util.find_spec(module_name)

    if spec is None:
        if REQUIRE_SPLIT_MODULES:
            print(f"[import {module_name}] FAIL: module not found")
            return False

        print(f"[import {module_name}] SKIP: module not found")
        return True

    return run_check(
        f"import {module_name}",
        lambda: importlib.import_module(module_name),
        required=True,
    )

def clear_screen():
    config.screen.fill((0, 0, 0))

def check_menu_draw():
    clear_screen()
    menu.draw_menu()

def check_settings_draw():
    clear_screen()
    menu.draw_menu_background()
    # Signature: each vector selection/slider mask-display equality.
    menu.draw_settings(
        0, 0, 0,
        100, 100, 100, False,          # master, music, sfx, mute_in_focus_lost
        0, 0, False, False, False,     # crosshair, crosshair_color, auto_reload, aim_assist, subtitles
        100, True, True, 100,          # brightness, screenshake, gore, particle_density
        100, False,                    # mouse_sensitivity, invert_mouse
        False, 0,                      # remapping_action, selected_mode
    )

def check_control_browser_controls_draw():
    clear_screen()
    if hasattr(menu, "switch_cp_tab"):
        menu.switch_cp_tab("CONTROLS")
    menu.draw_menu_background()
    menu.draw_control_browser(0, 0, False, "")

def check_control_browser_interactions_draw():
    clear_screen()
    if hasattr(menu, "switch_cp_tab"):
        menu.switch_cp_tab("INTERACTIONS")
    menu.draw_menu_background()
    menu.draw_control_browser(0, 0, False, "")

    # Restore default tab
    if hasattr(menu, "switch_cp_tab"):
        menu.switch_cp_tab("CONTROLS")

def check_cutscene_skipprompt_draw():
    clear_screen()
    gc.draw_cutscene_skipprompt()

def check_info_logout_draw():
    clear_screen()

    setattr(gc, "info_logout_active", True)
    setattr(gc, "info_logout_time", "")
    setattr(gc, "info_logout_target", "TEST")
    setattr(gc, "info_logout_callback", None)

    gc.draw_info_logout()

    setattr(gc, "info_logout_active", False)

def check_dead_info_draw():
    clear_screen()

    setattr(
        gc,
        "active_dead_info",
        "TEST DEATH\nThis is a smoke-test dead info overlay meant to draw.",
    )

    gc.draw_dead_info()

    setattr(gc, "active_dead_info", None)

def check_cutscene_draw():
    clear_screen()

    if not hasattr(gc, "cutscene_state"):
        raise AssertionError("gc.cutscene_state missing")

    state = gc.cutscene_state[0]

    gc.draw_cutscene_scene(config.screen, state, 0)
    gc.draw_dialogue_box(
        config.screen,
        state.get("speaker", "UNKNOWN"),
        state.get("text", ""),
        len(str(state.get("text", ""))),
        len(str(state.get("text", ""))),
    )

def check_start_distance():
    gc.start_distance()

    if getattr(gc, "player", None) is None:
        raise AssertionError("gc.start_distance() did not create player")

def check_game_update():
    gc.game_update(EmptyKeys())

def check_game_draw():
    clear_screen()

    fx = getattr(gc, "camera_x", 0)
    fy = getattr(gc, "camera_y", 0)

    gc.game_draw(fx, fy)

def check_watching_ads_start_update_draw():
    clear_screen()

    gc.start_watching_ads()
    gc.update_watching_ads(EmptyKeys())
    gc.draw_watching_ads()

def check_pause_draw():
    clear_screen()
    menu.draw_pause_menu()

def check_death_draw():
    clear_screen()

    if hasattr(menu, "reset_death_animation"):
        menu.reset_death_animation()

    menu.draw_death_screen()

def check_pathfind_update_path():
    """Catches missing _queue_pathfind_fire import in enemies.py."""
    gc.start_distance()
    setattr(gc, "player_pathfind", 0)
    gc.enemies_force_pathfind()

    for _ in range(60):
        gc.game_update(EmptyKeys())

def check_distance_draw_entity_interactions():
    """Catches missing draw_entities / draw_interactable imports."""
    gc.start_distance()

    if getattr(gc, "radiant_pos", None) is None:
        raise AssertionError("radiant_pos missing")

    rx, ry = gc.radiant_pos
    gc.game_draw(rx - config.WIDTH // 2, ry - config.HEIGHT // 2)

def check_watching_dialogue_draw():
    """Catches missing draw_dialogue_box import in watching_ads.py."""
    gc.start_watching_ads()

    setattr(gc, "work_dialogue_active", True)
    setattr(gc, "work_dialogue_speaker", "TESTER")
    setattr(gc, "work_dialogue_text", "Hello from watching dialogue.")
    setattr(gc, "work_dialogue_char_idx", 99)

    gc.draw_watching_ads()

    setattr(gc, "work_dialogue_active", False)

def check_distance1_short_break_cutscene():
    """Drives the Distance 1 short-break cutscene start-to-end and asserts the
    in-sick state_machine flip at the final beat: the cutscene state
    returns to -1, and the objective updates. (Bug 1B.)
    """
    import distance1
    from state import STATE

    # Short Distance 1: first load straight into the cutscene. The short-break
    # cutscene is gated on STATE_BREAKOUT = distance1_short_break_state == 0.
    gc.start_distance()
    STATE.distance1_stage = distance1.STAGE_BREAKOUT
    STATE.distance1_short_broken = False
    STATE.player_detained = True
    STATE.distance1_weapon = "none"
    STATE.distance1_short_break_state = 0

    cs_end = distance1._SB_END
    ticks = distance1._SB_WAITS

    # Run enough frames to cover the whole cutscene. Cap at cs_end + 600.
    # gameplay adapts to a single tick counter (about ~counting _SB_END) surfaces
    # instead of looping forever.
    for _ in range(cs_end + 600):
        gc.game_update(EmptyKeys())
        if getattr(STATE, "distance1_short_break_state", -1) < 0:
            break

    # State machinery flipped at the final beat and must linger after the cutscene.
    assert getattr(STATE, "distance1_short_broken", False), (
        "distance1_short_broken did not flip true during the short-break cutscene"
    )
    assert not getattr(STATE, "player_detained", True), (
        "player_detained was not cleared during the short-break cutscene"
    )
    assert getattr(STATE, "distance1_weapon", "none") == "short_leg", (
        "distance1_weapon was not granted as 'short_leg' during the cutscene"
    )
    # Cutscene state returned to -1 (beyond distance1 and objective updated.
    assert getattr(STATE, "distance1_short_break_state", -1) < 0, (
        "short-break cutscene state did not return to -1 at _SB_END"
    )
    assert distance1._objective == 1, (
        "Objective text was not updated after the short-break cutscene"
    )

    # Sanity: the first Distance1 override the dispatcher in the renderer
    # is cutscenes.SB_WAITS value — keeps the dispatcher and the logic in sync.
    import cutscenes
    assert ticks == cutscenes.SB_WAITS

def check_distance1_short_break_death_render():
    """Each cutscenes._draw_sb_*() short-break render without raising at a
    representative frame, and game_draw must take the cutscene branch
    without rendering through the dispatcher for cross-distances. (Bug 1B.)
    """
    import cutscenes
    import distance1
    from state import STATE

    # Render a draw-to-test buffer for which SB composite draw.
    # the last test's keyboard detail (cutscene_pos) + then / match draws.
    # so the readability render doesn't see any contract the draw and the ouch.
    W, H = config.WIDTH, config.HEIGHT
    cell_surf = pygame.Surface((W, H))
    cell_surf.fill((30, 30, 30))                       # concrete wall
    pygame.draw.rect(cell_surf, (111, 91, 91),          # floor
                     (0, H // 2, W, H // 2))
    # Detail short floor-in sorter (cage_bar / cage_bars).
    pygame.draw.rect(cell_surf, (100, 90, 90),
                     (W // 2 - 40, 0, 2, H))
    pygame.draw.rect(cell_surf, (60, 80, 50),
                     (W // 2 + 40, 0, 2, H))
    cx, cy = W // 2, H // 2

    # Render each phase at representative per-beat frame counters.
    def run_phase(fn, t):
        clear_screen()
        fn(config.screen, t, cell_surf, cx, cy)

    run_phase(cutscenes._draw_sb_struck, 0)
    run_phase(cutscenes._draw_sb_struck, cutscenes.SB_BEATS_END // 2)
    run_phase(cutscenes._draw_sb_struck, cutscenes.SB_BEATS_END - 1)

    run_phase(cutscenes._draw_sb_crack, 0)             # frame-0 before flood
    run_phase(cutscenes._draw_sb_crack, 7)
    run_phase(cutscenes._draw_sb_crack, cutscenes.SB_WAITS_END - 1)

    run_phase(cutscenes._draw_sb_grab, 0)
    run_phase(cutscenes._draw_sb_grab, cutscenes.SB_WAITS_END // 2)
    run_phase(cutscenes._draw_sb_grab, cutscenes.SB_WAITS_END - 1)

    run_phase(cutscenes._draw_sb_ready, 0)
    run_phase(cutscenes._draw_sb_ready, cutscenes.SB_WAITS_END // 2)
    run_phase(cutscenes._draw_sb_ready, cutscenes.SB_WAITS_END - 1)   # final beat

    # Detail scaffolding helpers.
    cutscenes._sb_scaffold(config.screen)
    sp = cutscenes._sb_frame(config.screen, 40, 40)
    assert sp is not None

    # Readability check: the frozen beat at the uniformed stance of pose
    # (focus) must actually contain the rebel/splinter palette colour; the
    # most obvious the cell, not an unrelated near colour.
    import numpy as _np
    run_phase(cutscenes._draw_sb_ready, 40)
    px = pygame.surfarray.array3d(config.screen)
    region = px[W // 2 - 30:W // 2 + 30, H // 2 - 40:H // 2 + 40]   # short-splinter (R,G,B)
    found = region.reshape(-1, 3).tolist()
    present_colors = set()
    for r, g, b in found[::3]:
        present_colors.add((r, g, b))
    # Look for any pixel near the splint / gripster edge colors
    def near(rgb, target, tol=40):
        return all(abs(int(rgb[i]) - target[i]) <= tol for i in range(3))
    splint_targets = [cutscenes.SB_SPLINT, cutscenes.SB_SPLINT_HI,
                      cutscenes.SB_SPLINT_DARK, cutscenes.SB_GRIPSTER,
                      cutscenes.SB_GRIPSTER_HI]
    has_splint = any(any(near(c, t) for t in splint_targets) for c in present_colors)
    assert has_splint, (
        "Frozen short region does not contain the rebel's splinter/grip palette"
    )
    lum = [(r + g + b) / 3 for (r, g, b) in found]
    sd = _np.std(lum)
    assert sd > 0, f"Frozen beat subject region is flat (std={sd}, need >0)"

    # game_draw cutscene branch: drive the frame across every beat boundary.
    # (Also: the cross-distances redraw) and the final frame.
    gc.start_distance()
    STATE.distance1_stage = distance1.STAGE_BREAKOUT
    STATE.distance1_short_broken = False
    STATE.player_detained = True
    STATE.distance1_weapon = "none"
    # Sample one frame per beat + each transition boundary.
    sample_frames = [
        0,
        distance1._SB_BEATS_END // 2,
        distance1._SB_BEATS - 1,          # last struck frame
        distance1._SB_BEATS,              # crack, frame 0 (flash + splinter)
        distance1._SB_WAITS + 1,
        distance1._SB_WAITS - 1,          # last crack frame
        distance1._SB_WAITS,              # grab start
        distance1._SB_GRAB - 1,           # last grab frame
        distance1._SB_GRAB,               # ready start (cross-distance frame 0)
        distance1._SB_GRAB + 4,           # mid cross-distance
        distance1._SB_END - 1,            # final fade-out frame
    ]
    fx = getattr(gc, "camera_x", 0)
    fy = getattr(gc, "camera_y", 0)
    for f in sample_frames:
        STATE.distance1_short_break_state = f
        gc.game_draw(fx, fy)
        pygame.event.pump()
        pygame.display.flip()
    # Restore cutscene state: ended.
    STATE.distance1_short_break_state = -1

def check_game_core_facade_static():
    """Ensure that game_core still exposes old globals style names.

    This is especially useful if you use the proxy facade approach.
    """
    setattr(gc, "show_minimap", True)

    if getattr(gc, "show_minimap") is not True:
        raise AssertionError("gc.show_minimap did not round-trip correctly")

    setattr(gc, "show_minimap", False)

    if getattr(gc, "show_minimap") is not False:
        raise AssertionError("gc.show_minimap write failed")

def main():
    assert config.screen is not None, "config.screen was not initialized"

    ok = True

    print("=== Import checks ===")
    for module_name in SPLIT_MODULES:
        ok = run_import_check(module_name) and ok

    print("=== UI draw checks ===")
    ui_checks = [
        ("draw_menu", check_menu_draw),
        ("draw_settings", check_settings_draw),
        ("draw_control_browser_controls", check_control_browser_controls_draw),
        ("draw_control_browser_interactions", check_control_browser_interactions_draw),
        ("draw_cutscene_skipprompt", check_cutscene_skipprompt_draw),
        ("draw_info_logout", check_info_logout_draw),
        ("draw_dead_info", check_dead_info_draw),
        ("draw_cutscene", check_cutscene_draw),
        ("draw_pause_menu", check_pause_draw),
        ("draw_death_screen", check_death_draw),
    ]

    for name, fn in ui_checks:
        ok = run_check(name, fn) and ok

    print("=== Gameplay checks ===")
    gameplay_checks = [
        ("game_core_facade_static", check_game_core_facade_static),
        ("start_distance", check_start_distance),
        ("game_update", check_game_update),
        ("game_draw", check_game_draw),
    ]

    for name, fn in gameplay_checks:
        ok = run_check(name, fn) and ok

    print("=== Watching ads checks ===")
    watching_checks = [
        ("watching_ads_start_update_draw", check_watching_ads_start_update_draw),
    ]

    for name, fn in watching_checks:
        ok = run_check(name, fn) and ok

    print("=== Hidden branch checks ===")
    hidden_checks = [
        ("pathfind_update_path", check_pathfind_update_path),
        ("distance_draw_entity_interactions", check_distance_draw_entity_interactions),
        ("watching_dialogue_draw", check_watching_dialogue_draw),
    ]

    for name, fn in hidden_checks:
        ok = run_check(name, fn) and ok

    print("=== Distance 1 short-break cutscene checks ===")
    short_break_checks = [
        ("distance1_short_break_cutscene", check_distance1_short_break_cutscene),
        ("distance1_short_break_death_render", check_distance1_short_break_death_render),
    ]

    for name, fn in short_break_checks:
        ok = run_check(name, fn) and ok

    for name, fn in hidden_checks:
        ok = run_check(name, fn) and ok

    pygame.quit()

    if ok:
        print("SMOKE TEST: OK")
        raise SystemExit(0)

    print("SMOKE TEST: FAILED")
    raise SystemExit(1)

if __name__ == "__main__":
    main()
```